

Descriptif du module

Filière : Systèmes industriels

Orientation : Infotronics

Axe d'enseignement : Orientation Infotronics

Type de formation : B.Sc.

Année académique : 2026-27

Description du Module						
Module	Software Development 1 (SWD1)					
Type	Module obligatoire	Numéro	I3.2	ECTS crédits	7	
Langues	Français - Allemand	Semestre (S)	Plein temps	3	Temps partiel	5
Organisation temporelle	Semestre d'automne					
Responsable de module	Silvan Zahno (zas)					

Contenu du Module								
Numéro	Unités d'enseignement (UE)	Forme	Semestre					
			1	2	3	4	5	6
I3.21	Advanced Programming (AProg)	Cours et exercices en classe et travaux en laboratoire			x			
I3.22	Applied Programming (ProgA)	Projet de développement logiciel encadré			x			

x : cours obligatoire ; c : cours à choix ; o : cours optionnel ; (.) en continuité dans un autre module

Prérequis	
Avoir validé S1.3 (BaM)	Ce module requière les connaissances de base nécessaires dans le domaine de la programmation et de la mise en œuvre d'algorithmes.

Compétences visées / Objectifs généraux d'apprentissage

Les objectifs d'apprentissage de ce module, spécifique à l'orientation Infotronics, visent à développer les compétences nécessaires à la conception, au développement, aux tests et à la maintenance d'applications logicielles modernes. Le module met l'accent sur les paradigmes de programmation actuels, les bonnes pratiques du génie logiciel, la qualité du code ainsi que la réalisation d'un projet logiciel complet.

- Comprendre et appliquer les paradigmes de programmation impérative, orientée objet et générique dans le développement logiciel moderne (C)
- Maîtriser les concepts fondamentaux du langage Rust, notamment la gestion sûre de la mémoire, l'ownership, les emprunts (borrowing), les traits, les génériques et les collections (A)
- Concevoir des logiciels robustes, modulaires et maintenables en appliquant les principes du génie logiciel, les patrons de conception (Design Patterns) et les bonnes pratiques de développement (A, J)
- Développer des applications concurrentes et événementielles, en utilisant les mécanismes modernes de communication, de parallélisme et d'asynchronisme (A)
- Mettre en œuvre des méthodes garantissant la qualité logicielle, notamment les tests unitaires, la documentation, le débogage, la gestion de versions et les outils de développement modernes (A)
- Intégrer les connaissances acquises dans le cadre d'un projet d'ingénierie logicielle complet, depuis l'analyse du problème jusqu'à l'implémentation, aux tests et à la documentation de la solution (A, J)

C : Connaissance et compréhension

A : Application des connaissances et de la compréhension

J : Capacité de former des jugements (analyse, évaluation)

Cadre de qualification : référence 1

Modalités de validation du module

Contrôles continus et examens semestriels :

- Le contrôle continu comprend les exercices, les laboratoires, les rapports, les présentations, épreuves orales ou écrites effectués durant le semestre ainsi que les codes dans les dépôts Git.
- Le nombre de contrôles continus et leur pondération relèvent de la compétence des professeurs
- Les notes du contrôle continu et les notes des examens semestriels sont attribuées au dixième de point

La note finale du module est calculée au demi-point en tenant compte des pondérations indiquées dans le tableau. Le module est validé si la note finale du module est d'au moins 4.

Numéro	Unité d'enseignement	Semestre	Pondération		
			Contrôle continu	Examens semestriels	Poids de l'UE
I3.21	Advanced Programming	S3	1	2	2
I3.22	Applied Programming	S3	1		1

Modalités de remédiation

La remédiation de ce module est autorisée dans le respect des dispositions réglementaires en vigueur². L'épreuve de remédiation pourra consister soit en un examen (écrit ou oral), soit en un travail complémentaire. Aucune remédiation n'est possible en cas de répétition du module.

Modalités de répétition

L'étudiante ou l'étudiant qui répète un module refait les unités d'enseignement du module dont la moyenne est strictement inférieure à 4,8. La nouvelle note obtenue fera référence pour le calcul de la note finale du module répété.

Les laboratoires et/ou projets dont la moyenne est égale ou supérieure à 4,8 n'ont pas l'obligation d'être répétés. La note initialement obtenue fera référence pour le calcul de la moyenne finale de l'UE répétée. Si les laboratoires ne sont pas notés, le ou la responsable de l'UE prend position sur la nécessité ou non de répéter les laboratoires.

Remarques

La présence aux travaux pratiques est obligatoire. En cas d'absence dûment constatée, la personne responsable de l'unité d'enseignement peut exiger la réalisation d'un travail complémentaire afin de pouvoir se présenter aux examens de semestre.

Validation – Responsable de filière

Date : 14.09.2026

Nom : Bezençon Cyrille (bcy)

Légende, références

1. *Swissuniversities, Cadre de qualifications pour le domaine des hautes écoles suisses nqf.ch-HS
C : Connaissance et compréhension
A : Application des connaissances et de la compréhension
J : Capacité de former des jugements (analyse, évaluation)
SC : Savoir-faire en termes de communication
AA : Capacités d'apprentissage en autonomie
2. Directive DI.1.2.01.02 F
Directive relative à l'évaluation et à la validation des modules des filières Bachelor de la Haute Ecole d'Ingénierie

Unité d'enseignement (UE) – Advanced Programming (I3.21)

Objectifs généraux d'apprentissage (Compétences visées)

A la fin de l'unité d'enseignement, l'étudiante ou l'étudiant est capable de :

- Décrire les principaux paradigmes de programmation modernes ainsi que leurs domaines d'application. (C)
- Connaître et utiliser les principaux langages de programmation procéduraux et orientés objets (A)
- Maîtriser les concepts fondamentaux du langage Rust, notamment l'ownership, les emprunts (borrowing), les durées de vie (lifetimes), les traits, les génériques et les collections. (A)
- Développer des applications modulaires, robustes et maintenables en appliquant les principes du génie logiciel et les principaux patrons de conception (Design Patterns). (A)
- Développer des applications événementielles, concurrentes et asynchrones en utilisant les mécanismes modernes de communication et de synchronisation. (A)
- Connaître les principaux environnements de développement (IDE) actuels (C)
- Utiliser les outils modernes de développement logiciel, notamment les environnements de développement, Git, les tests unitaires, le débogage et la documentation. (A)
- Évaluer la qualité, la maintenabilité et la pertinence des solutions logicielles développées au regard des bonnes pratiques du génie logiciel. (J)

C : Connaissance et compréhension

A : Application des connaissances et de la compréhension

J : Capacité de former des jugements (analyse, évaluation)

Cadre de qualification : référence 1

Contenu de l'unité d'enseignement

Thèmes	Description brève
Introduction	Présentation du langage Rust, de son écosystème, des outils de développement (Zed, Git, Cargo) et des principes fondamentaux du développement logiciel moderne.
Programmation impérative	Variables, système de types, fonctions, structures de contrôle, ownership, borrowing, lifetimes, gestion des erreurs et collections.
Programmation orientée objet	Structs, enums, implémentations, traits, génériques, smart pointers, organisation modulaire et principes de conception orientée objet en Rust.
Patrons de conception	Étude et mise en œuvre d'une sélection de patrons de conception adaptés à Rust : Singleton, NewType, RAII, Builder et Strategy.
Programmation concurrente	Threads, communication par canaux, synchronisation, programmation asynchrone (async/await) et développement d'applications réactives.
Débogage, tests et documentation	Débogage, tests unitaires, documentation, gestion de versions avec Git et bonnes pratiques pour développer des logiciels robustes et maintenables.

Unité d'enseignement (UE) – Applied Programming (I3.22)

Objectifs généraux d'apprentissage (Compétences visées)

A la fin de l'unité d'enseignement, l'étudiante ou l'étudiant est capable de :

- Analyser un cahier des charges et concevoir l'architecture d'une application logicielle répondant à des exigences fonctionnelles et techniques. (C)
- Développer une application complète en Rust en appliquant les principes du génie logiciel et les bonnes pratiques de programmation. (A)
- Mettre en œuvre des stratégies de validation fondées sur les tests, l'intégration continue et la gestion de versions avec Git. (A)
- Réaliser les différentes étapes d'un projet logiciel selon des jalons définis, de l'analyse jusqu'à la livraison d'une application fonctionnelle. (A)
- Évaluer la qualité d'une solution logicielle au regard de critères de robustesse, de maintenabilité, de documentation et d'interopérabilité. (J)

C : Connaissance et compréhension

A : Application des connaissances et de la compréhension

J : Capacité de former des jugements (analyse, évaluation)

Cadre de qualification : référence 1

Contenu de l'unité d'enseignement

Thèmes	Description brève
Analyse des besoins	Lecture et analyse d'un cahier des charges, compréhension des exigences fonctionnelles et techniques, définition de l'architecture de la solution.
Développement d'une application	Réalisation d'une application complète en Rust en appliquant les connaissances acquises dans le cours Advanced Programming.
Validation et tests	Développement piloté par les tests, validation progressive, intégration continue et vérification de l'interopérabilité de l'application.
Gestion de projet logiciel	Organisation du développement par jalons, gestion de versions avec Git, documentation, suivi des livrables et gestion des évolutions du logiciel.
Qualité logicielle	Revue de code, qualité du code, robustesse, maintenabilité, documentation et démonstration finale de la solution développée.

Référence bibliographique et support de cours

- Advanced Programming, Silvan Zahno, 2026
- The Rust Programming Language - S. Klabnik, C. Nichols, and C. Kryocho
- J. Blandy, J. Orendorff, and L. F. s Tindall, Programming Rust: Fast, Safe Systems Development. O'Reilly Media, Incorporated, 2021.
- H. Collard, Black Hat Rust, vol. 1. Amazon, 2025.
- A. Shvets, "Refactoring Guru." 2026.
- R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Pearson Education, 2008.

Remarques sur l'unité d'enseignement

Validation – Responsable de l'Unité d'enseignement

Date : 14.09.2026

Nom : Silvan Zahno (zas)